

Abstract Syntax

Chapter 4

4.2 Abstract Parse Trees

- It is possible to write an entire compiler that fits **within the semantic action phrases** of a Yacc parser.
 - difficult to read and maintain
 - must analyze the program in exactly the order it is parsed

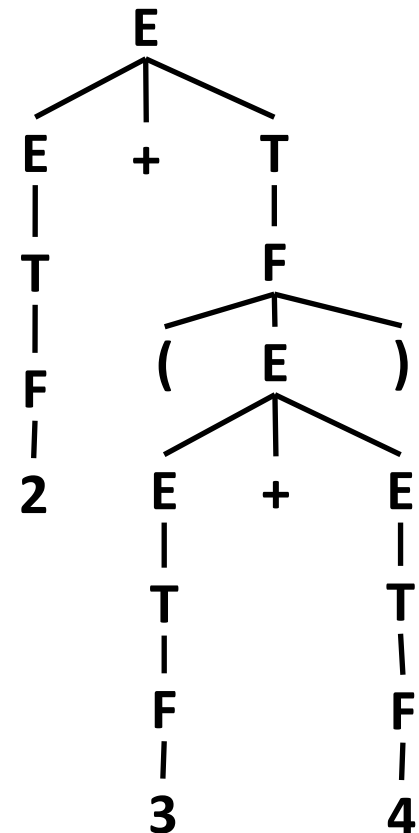
```
void foo() { bar(); }  
void bar() { ... }
```

- To improve modularity, it is better to separate **issues of syntax** (parsing) from **issues of semantics** (type-checking and translation to machine code)
- **One solution**: the parser produces a parse tree that later phases can traverse.

Abstract Syntax: Why

- Technically, a **parse tree** has exactly **one leaf for each token** of the input and **one internal node for each grammar rule** reduced during the parse.
- Such a parse tree is called a **concrete parse tree**, representing the concrete syntax of the source language.
- A concrete parse tree is **inconvenient** to use directly:
 - redundant and useless tokens for later phases
 - e.g., **(,)**
 - memory usage
 - depends too much on the grammar
 - The grammar changes $\rightarrow$ parse tree changes

$E \rightarrow E + T$
| T
 $T \rightarrow T * F$
| F
 $F \rightarrow n \mid (E)$



Abstract Syntax: What

- An **abstract syntax** makes a clean interface between the parser and the later phases of a compiler.
- The abstract syntax **is not meant for parsing**.
- The parser uses the **concrete syntax** to build a parse tree for the abstract syntax — **abstract syntax tree**

E -> **E** + **T**

| **T**

T -> **T** * **F**

| **F**

F -> **n** | (**E**)

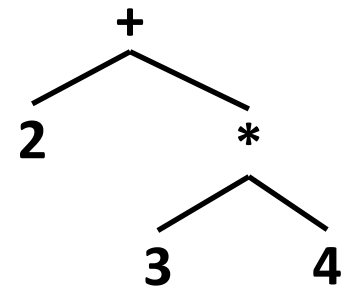
Concrete Syntax

E -> **n**

| **E** + **E**

| **E** * **E**

Abstract Syntax



Abstract Syntax Tree

- The **abstract syntax tree** conveys the **phrase structure** of the source program, with all parsing issues resolved but without any semantic interpretation.

Abstract Syntax Tree

- To use abstract syntax tree in later phases, the compiler will need to represent and manipulate abstract syntax trees as **data structures**. **How?**
- A typedef for each nonterminal, a union-variant for each production.

```
typedef struct A_exp_ *A_exp;  
struct A_exp_ {  
    enum {A_numExp, A_plusExp, A_timesExp} kind;  
    union {  
        int num;  
        struct {A_exp left; A_exp right;} plus;  
        struct {A_exp left; A_exp right;} times;  
    } u;  
};
```

```
A_exp A_NumExp(int num);  
A_exp A_PlusExp(A_exp left, A_exp right);  
A_exp A_TimesExp(A_exp left, A_exp right);
```

$E \rightarrow n$
$E + E$
$E * E$

Abstract Syntax Tree

- To use abstract syntax tree in later phases, the compiler will need to represent and manipulate abstract syntax trees as **data structures**. **How?**
- A typedef for each nonterminal, a union-variant for each production.

```
A_exp A_PlusExp(A_exp left, A_exp right) {  
    A_exp e = checked_malloc(sizeof(*e));  
    e->kind = A_plusExp;  
    e->u.plus.left = left;  
    e->u.plus.right = right;  
    return e;  
}
```

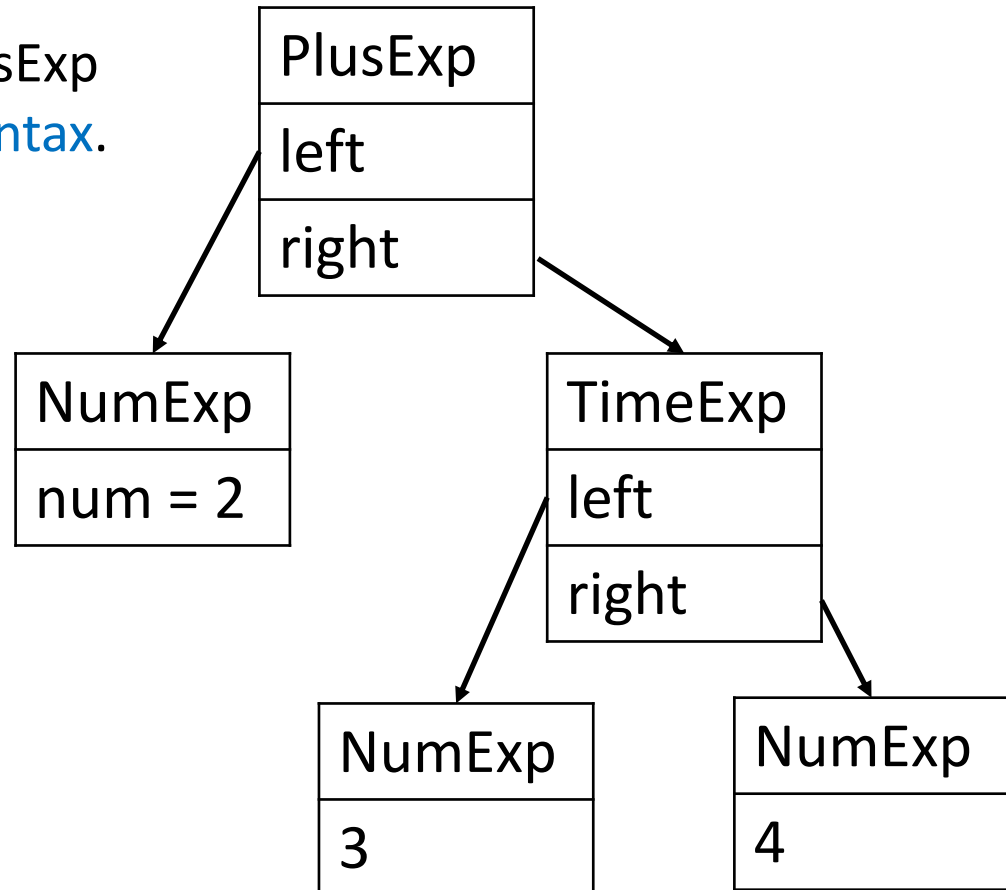
Abstract Syntax Tree

- Construct the abstract syntax tree of $2 + 3 * 4$ using these data structures

We know we should create TimesExp first according to the **concrete syntax**.

```
e1 = A_NumExp(2);  
e2 = A_NumExp(3);  
e3 = A_NumExp(4);  
e4 = A_TimesExp(e2, e3);  
e5 = A_PlusExp(e1, e4);
```

We did not evaluate.



Abstract Syntax Tree

- Can we construct abstract syntax trees automatically?
- The Yacc-generated (or recursive-descent) parser, parsing the concrete syntax, constructs the abstract syntax tree.
- How?

```
%left PLUS  
%left TIMES
```

```
%%  
exp : NUM { $$=A_NumExp($1); }  
    | exp PLUS exp { $$=A_PlusExp($1, $3); }  
    | exp TIMES exp { $$=A_TimesExp($1, $3); }
```

Positions

- In a one-pass compiler, lexical analysis, parsing, and semantic analysis are all done simultaneously.
- If there is a **type error** that must be reported to the user, the **current position** of the lexical analyzer is a reasonable approximation of the source position of the error.
- In a **one-pass** compiler, the lexical analyzer keeps a “**current position**” global variable.
- For a compiler that **uses abstract-syntax-tree data structures**:
 - It need not do all the parsing and semantic analysis in one pass
 - The lexer reaches the end of file before semantic analysis even begins.
- How can we know the error position if there is a type error?

Positions

- The source-file position of each node of the abstract syntax tree must be remembered.
- How?
- The abstract-syntax data structures must be sprinkled with **pos** fields, which indicate the **position**, within the original source file, of the characters **from which these abstract syntax structures were derived**.
- How to set the **pos** fields?
- First, the **lexer** must pass the positions of the beginning and end of each token to the **parser**
- Then, for parsers:

Positions

- Ideally, the parser should maintain a *position stack* along with the *semantic value stack*, making the position of each symbol *available for the semantic actions to use*.
 - Bison can do this, **Yacc does not**
- **Yacc**: one solution is to *define a nonterminal symbol pos* whose semantic value is a *source location* (line number, or line number and position within line).
 - e.g., access the position of the **PLUS**

```
%{ extern A_0pExp (A_exp,A_binop,A_exp,position); %}  
%union { int num; string id; position pos;...};  
%type <pos> pos
```

```
pos: { $$ = EM_tokpos; }  
exp: exp PLUS pos exp {$$= A_0pExp($1, A_plus, $4, $3); }
```

Homework

- **4.2**